Classical AI Reasoning System

Implementation and Evaluation

AMAN KUMAR

Roll No.: M26MAC001

Overview

This report presents a system that checks whether a query follows from a knowledge base of facts and rules. Inputs are converted into logical clauses and processed using forward chaining, backward chaining, resolution and DPLL. The system records reasoning steps and identifies unresolved inputs. Evaluation on seven public dataset files containing 4,029 queries produced 3,618 correct predictions, giving an overall binary accuracy of 89.80%.

1 Knowledge Representation and Query Processing

1.1 Facts and Logical Formulas

A term represents a constant, a variable or a function application. An atom is a predicate applied to terms, and a literal is an atom or its negation. For example, $\mathsf{Young}(bob)$ represents a fact, while

$$\forall x \left(\mathsf{Young}(x) \rightarrow \mathsf{Smart}(x)\right)$$

represents a rule. A query such as $\mathsf{Smart}(bob)$ asks whether the statement follows from the knowledge base.

Formulas are stored as trees so that their operators and arguments can be processed separately. The parser accepts formal logic and a limited set of English fact and rule patterns. FOLIO inputs use the formal formulas supplied in the dataset.

1.2 Clause Conversion and Entailment

Formulas are converted into conjunctive normal form (CNF): a conjunction of clauses, each containing a disjunction of literals. For example, $A \rightarrow B$ becomes $\neg A \vee B$.

Conversion removes implications, moves negations to atoms and renames bound variables. Existential variables are replaced with fresh constants or functions through Skolemization. Function-free clauses are grounded over the constants in the problem.

For a query q, the system uses the entailment test

$$\mathrm{KB} \models q \quad \Longleftrightarrow \quad \mathrm{KB} \wedge \neg q \text{ is unsatisfiable.}$$

True means that the query is entailed. False means that it is not entailed; this does not necessarily prove its negation. Invalid inputs and searches that reach a resource limit are recorded as unresolved.

1.3 Representing CounterBench Inputs

CounterBench inputs describe Boolean equations, observations and interventions. The parser creates factual and hypothetical copies of the variables, sharing the same background variables. An intervention replaces the relevant hypothetical equation.

DPLL checks whether the observations are consistent with the equations and then tests the equations with the negated query. The representation assumes acyclic, deterministic Boolean equations.

2 Inference Methods

2.1 Forward Chaining: Starting from Facts

Forward chaining derives new facts by applying rules to known facts. It is used for ground Horn clauses, which contain at most one positive literal. A clause without a positive literal acts as a constraint.

1. Count the unproved premises of each rule.
2. Place rules with no remaining premises in a queue.
3. Record each new conclusion and update dependent rules.
4. Stop when a constraint is contradicted or no further rule can be applied.

The queue avoids repeatedly checking every rule. After grounding, propagation is linear in the number of literal occurrences.

2.2 Backward Chaining: Starting from a Goal

Backward chaining begins with a goal and searches for rules whose conclusions match it. Their premises become subgoals. A fact proves a subgoal directly; a rule proves it when all its premises are proved.

Rules are indexed by their conclusions. Active goals are recorded to stop repeated traversal of cycles, and proved goals are cached. A failed cyclic branch is not permanently cached as false because another rule may prove the goal. Proving every premise of a constraint establishes a contradiction in $\mathrm{KB} \wedge \neg q$.

2.3 Resolution: Proving a Contradiction

Resolution combines clauses containing complementary literals:

$$\frac{A \vee B \qquad \neg B \vee C}{A \vee C}.$$

For first-order clauses, unification finds substitutions that make the complementary literals match. Variables are renamed between clauses, and an occurs check rejects substitutions that place a variable inside its own value.

Factoring combines unifiable literals within a clause. The solver records parent clauses for each result. Deriving the empty clause, $\square$, proves a contradiction. Time and clause limits bound the search.

2.4 DPLL: Checking Satisfiability

Davis–Putnam–Logemann–Loveland (DPLL) searches for a satisfying assignment to Boolean clauses.

1. Detect success when all clauses are satisfied, or failure when a clause is empty.
2. Use a unit clause to assign its only remaining literal.
3. Assign a pure literal, which occurs with only one sign.
4. Branch on a remaining literal and backtrack if the chosen value fails.

A satisfying assignment to $\mathrm{KB} \wedge \neg q$ is a counterexample to the query. If the completed search finds no satisfying assignment, the query is entailed.

2.5 Choosing an Inference Method

Clauses with function terms use resolution. Other clauses are grounded. Ground Horn clauses with one constraint use backward chaining; those with zero or several constraints use forward chaining. Other Boolean clauses and CounterBench equations use DPLL. The chaining method can also be selected explicitly.

3 Dataset Evaluation and Accuracy

3.1 Evaluation Settings

Seven files from RuleTaker, CounterBench and FOLIO were evaluated [1, 2, 3]. Records are processed without their reference labels, IDs or stored explanations. Reference labels are used only after prediction to calculate accuracy:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Labelled queries}} \times 100.$$

For binary entailment, FOLIO's True label maps to True; False and Uncertain map to False. The run used Python 3.12.14 on Windows with a one-second reasoning budget per query. Times include parsing and inference but exclude file loading. These are public development results; dataset formats and errors were inspected during implementation.

Unresolved records return `None` internally. The CSV exports False for these records. All reported accuracy values include these fallback predictions; unresolved counts are shown separately. A fallback prediction is not a completed logical deduction.

3.2 RuleTaker Results

RuleTaker supplies facts and rules for testing whether a query follows from a knowledge base. Backward chaining was selected for all RuleTaker records in this run.

Split	Queries	Correct	Accuracy	Unresolved	Time (s)
Train	1,000	983	98.30%	0	1.341
Test	250	244	97.60%	0	0.364
OOD test	375	373	99.47%	0	0.500
Combined	**1,625**	**1,600**	**98.46%**	**0**	**2.205**

Table 1: RuleTaker accuracy and execution time. OOD denotes the out-of-distribution split.

Train. The system correctly answered 983 of 1,000 queries, giving 98.30% accuracy. All records were resolved, with 17 incorrect predictions.

Test. Accuracy was 97.60%, with 244 correct predictions out of 250. The six incorrect predictions were completed inference results; no record reached an input or search failure.

OOD test. Accuracy was 99.47%, with 373 correct predictions out of 375. This was the highest split accuracy in the evaluation. All records were resolved.

Across the three files, the combined accuracy was 98.46%. The results show high accuracy on the evaluated rule-based inputs.

3.3 CounterBench Results

CounterBench tests queries involving Boolean equations, observations and interventions. The equations are encoded as clauses and checked using DPLL.

Variant	Queries	Correct	Accuracy	Unresolved	Time (s)
Alpha	1,000	905	90.50%	165	3.545
Backdoor	200	175	87.50%	10	0.617
Combined	**1,200**	**1,080**	**90.00%**	**175**	**4.162**

Table 2: CounterBench accuracy and execution time.

Alpha. The exported predictions were correct for 905 of 1,000 queries, giving 90.50% accuracy. There

were 165 unresolved records: 155 were reported as inconsistent with the parsed equations, and 10 produced input errors.

Backdoor. The exported predictions were correct for 175 of 200 queries, giving 87.50% accuracy. Ten records were unresolved: five were reported as inconsistent with the parsed equations, and five produced input errors.

The combined accuracy was 90.00%. Alpha had higher accuracy, while Backdoor had a smaller proportion of unresolved records. Both accuracy and unresolved counts are therefore needed to assess the results.

3.4 FOLIO Results

FOLIO supplies first-order formulas involving logical connectives and quantifiers. The system reads the supplied formal premises and conclusion, then selects an inference method from the clause structure.

Split	Queries	Correct	Accuracy	Unresolved	Time (s)
Train	1,001	774	77.32%	234	44.547
Validation	203	164	80.79%	62	3.738
Combined	**1,204**	**938**	**77.91%**	**296**	**48.285**

Table 3: FOLIO accuracy under binary entailment evaluation.

Train. The exported predictions were correct for 774 of 1,001 queries, giving 77.32% accuracy. There were 234 unresolved records: 205 input errors and 29 searches that reached a resource limit. This file had the longest execution time, at 44.547 seconds.

Validation. The exported predictions were correct for 164 of 203 queries, giving 80.79% accuracy. There were 62 unresolved records: 57 input errors and five searches that reached a resource limit.

The combined accuracy was 77.91%. FOLIO had lower accuracy and more unresolved records than the other dataset families. The recorded input errors and resource limits identify limitations in formula handling and inference coverage.

3.5 Combined Performance

Dataset	Queries	Correct	Accuracy	Unresolved	Time (s)
RuleTaker train	1,000	983	98.30%	0	1.341
RuleTaker test	250	244	97.60%	0	0.364
RuleTaker OOD	375	373	99.47%	0	0.500
CounterBench alpha	1,000	905	90.50%	165	3.545
CounterBench backdoor	200	175	87.50%	10	0.617
FOLIO train	1,001	774	77.32%	234	44.547
FOLIO validation	203	164	80.79%	62	3.738
Total	**4,029**	**3,618**	**89.80%**	**471**	**54.652**

Table 4: Results from the complete notebook run. OOD denotes the out-of-distribution split.

Across all seven files, 3,618 of 4,029 exported predictions were correct, giving an overall accuracy of **89.80%**. There were 471 unresolved records, and total parsing and inference time was 54.652 seconds.

Combined accuracies are calculated from total correct predictions divided by total queries, rather than by averaging split percentages. The CSV contains all 4,029 predictions with unique IDs. Runtime and search-limit outcomes can vary with the computer and its workload.

4 Implementation Checks

The notebook contains 16 basic checks and examples for all four methods. Development validation passed 43 tests and 1,180 generated checks covering new symbol names, reordered rules, removed facts, cyclic dependencies and irrelevant labels or IDs. Small Boolean cases were also checked against independent truth tables.

5 Illustrative Reasoning Traces

5.1 Derivation by Forward Chaining

Consider the facts $\mathsf{Young}(bob)$ and $\mathsf{Kind}(bob)$, with the rules

$$\mathsf{Young}(x) \to \mathsf{Smart}(x),$$
$$\mathsf{Smart}(x) \wedge \mathsf{Kind}(x) \to \mathsf{Successful}(x).$$

For the query $\mathsf{Successful}(bob)$:

1. Derive $\mathsf{Smart}(bob)$ from $\mathsf{Young}(bob)$.
2. Combine $\mathsf{Smart}(bob)$ and $\mathsf{Kind}(bob)$ to derive $\mathsf{Successful}(bob)$.
3. Contradict the added literal $\neg\mathsf{Successful}(bob)$ and return True.

5.2 Goal Reduction by Backward Chaining

Using the same knowledge base, begin with $\mathsf{Successful}(bob)$. Its rule requires $\mathsf{Smart}(bob)$ and $\mathsf{Kind}(bob)$. The second subgoal is a fact. The first requires $\mathsf{Young}(bob)$, which is also a fact. All subgoals are proved, so the result is True.

5.3 Refutation by Resolution

Let the knowledge base contain $P(a)$ and $\forall x\,(P(x) \to P(f(x)))$, with query $P(f(f(a)))$. The recorded proof contains the following clauses; its generated variable is written as x for readability.

Clause	Formula	Origin
C_1	$P(a)$	Input fact
C_2	$\neg P(x) \vee P(f(x))$	Input rule
C_3	$\neg P(f(f(a)))$	Negated query
C_4	$P(f(a))$	Resolve C_2 with C_1
C_5	$\neg P(f(a))$	Resolve C_2 with C_3
C_9	$\square$	Resolve C_5 with C_4

The empty clause proves a contradiction, so the result is True.

5.4 Counterexample Found by DPLL

For KB $= A \vee B$ and query A, test $(A \vee B) \wedge \neg A$. Unit propagation sets $A =$ False and then $B =$ True. This assignment satisfies the knowledge base and makes the query false. It is a counterexample, so the result is False.

6 Scope and Limitations

The English parser supports a limited grammar. Unsupported or malformed inputs may remain unresolved. CNF conversion and grounding can substantially increase the number of clauses. DPLL may require exponential search, and general first-order resolution may not terminate, so resource limits are applied.

Equality is supported for function-free formulas; equality involving function terms is unsupported. Binary entailment does not distinguish a disproved query from a query whose truth is unknown. Under classical logic, an inconsistent knowledge base entails every query.

References

[1] jise/ruletaker. *RuleTaker dataset variant*. `https://huggingface.co/datasets/jise/ruletaker`.

[2] CounterBench. *Public dataset*. `https://huggingface.co/datasets/CounterBench/CounterBench`.

[3] Han et al. (2024). *FOLIO: Natural Language Reasoning with First-Order Logic*. EMNLP. `https://aclanthology.org/2024.emnlp-main.1229/`.